

Building q-Quiz: A Practice Platform for kdb+/q

A HackerRank-style learning site, graded by a real live q process instead of pattern matching.

Spencer Pruno · kdb+/q & real-time analytics

kdb+/q is one of those languages you either love or have never heard of. It quietly runs the market-data infrastructure behind a huge amount of the buy side and sell side — real-time tick capture, historical analytics, risk engines — and yet there's a real shortage of places to actually practice it hands-on. You can read the reference manual, or you can get thrown at a production kdb+ system. There isn't much in between.

So I built **q-Quiz**: a self-hosted, HackerRank-style practice platform built specifically around q, where every question and every submission is graded by a live q process — not string matching, not a mocked judge. Here's a tour of what's in it so far.

Fundamentals: Multiple Choice and qldioms

The starting point is a multiple-choice bank covering core q concepts — attributes, joins, tables, IPC, qsql, and more — split into a general-concepts track and a dedicated syntax track (right down to catching the difference between `each`, `over`, and `scan`). Underneath that, qldioms and Fundamentals push further with short, idiom-style codepad exercises.

q-Quiz

HomeResults

Sign inSpencer Pruno

Question 798

Why are equality filters on enumerated columns generally faster than on strings?

A The sym file is always compressed

B Symbols are automatically cached

C Integer comparisons are cheaper than string comparisons

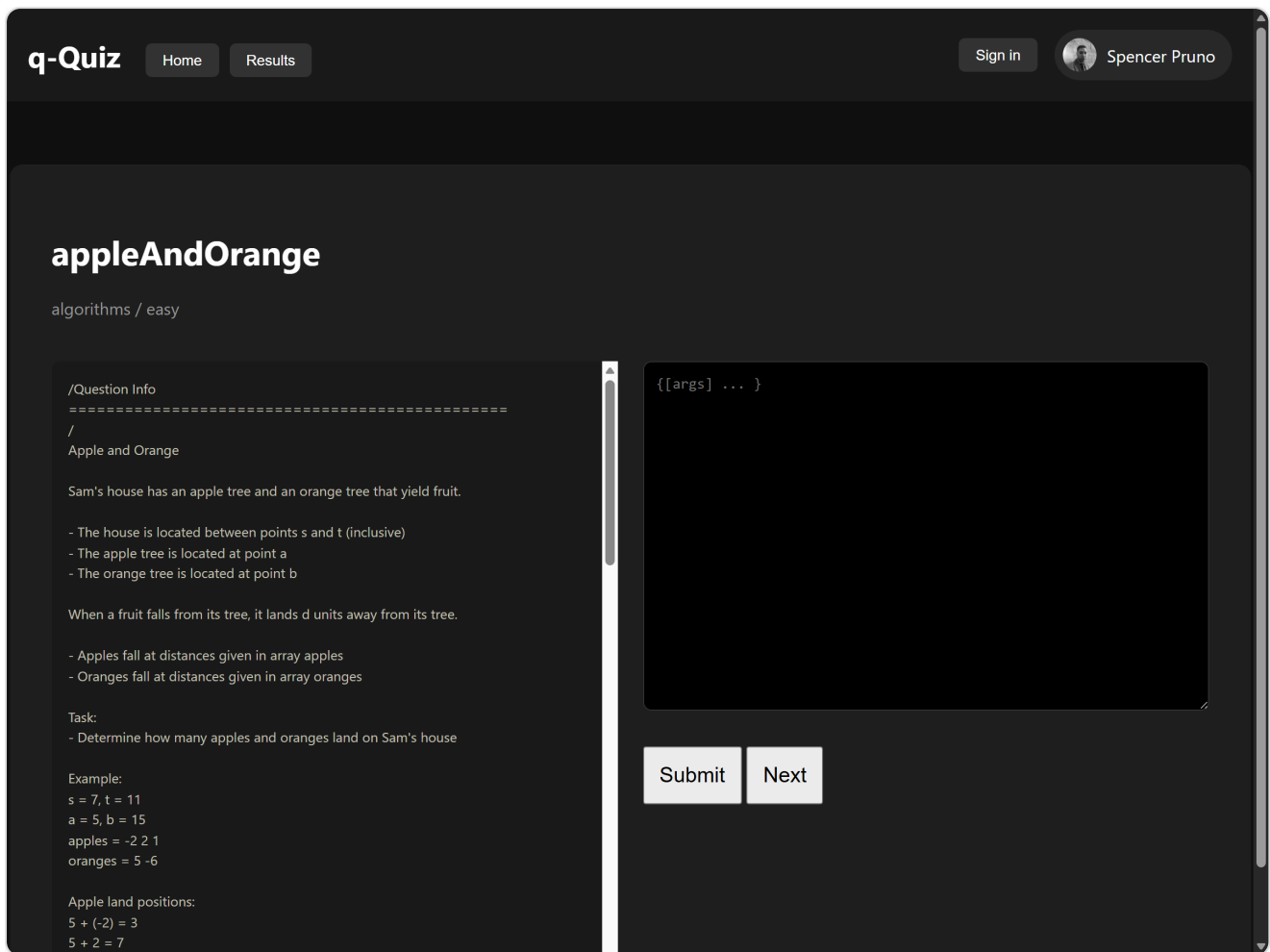
D Enumerated columns are sorted

Submit

The Multiple Choice quiz, drawing from topic-organized question banks.

Challenges: HackerRank, AquaQ, LeetCode, quantRank

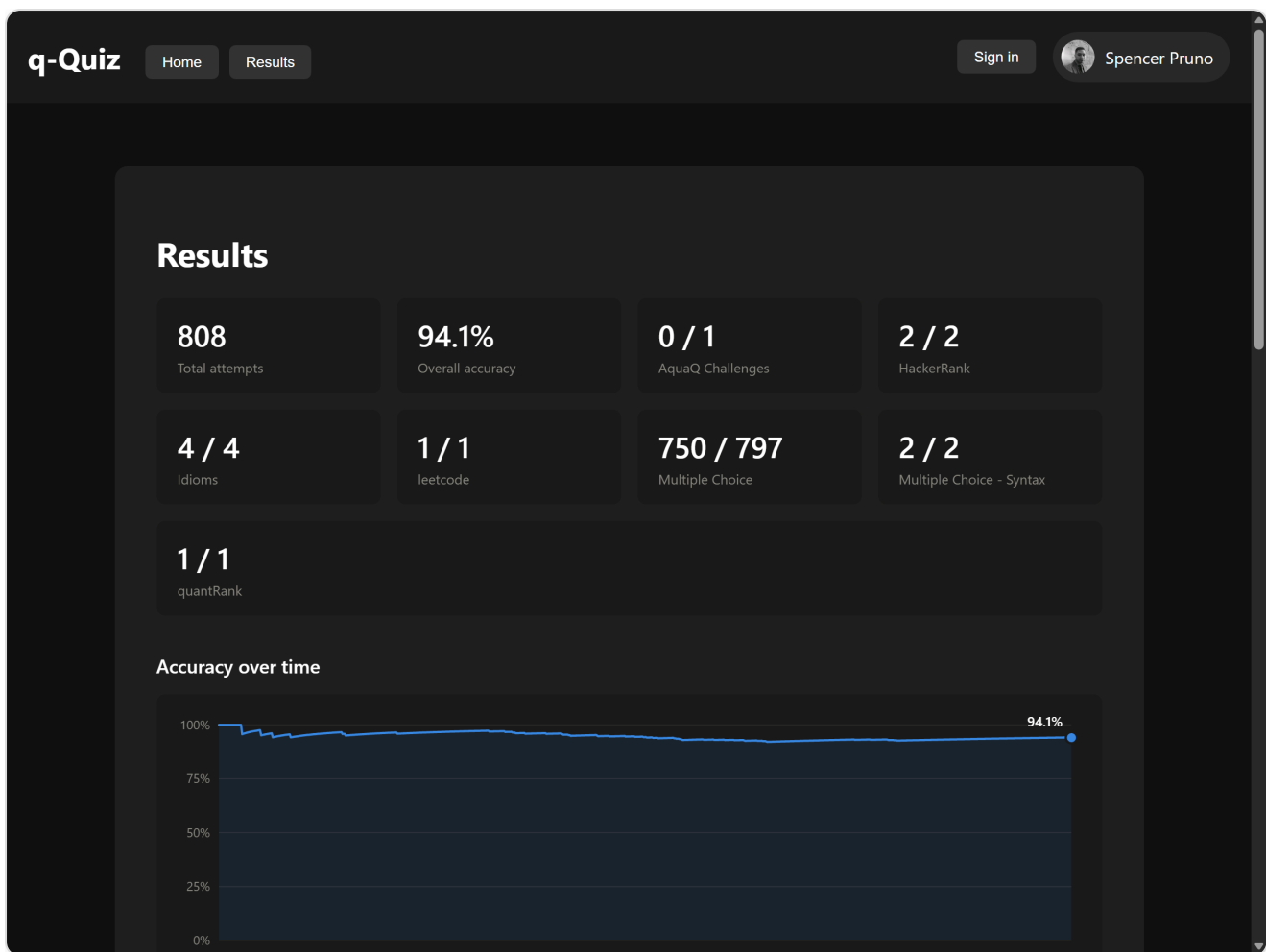
For deeper practice there are four codepad-based challenge sets — HackerRank-style algorithm problems, AquaQ's published interview challenges, LeetCode problems translated into q, and quantRank problems geared at quant-specific reasoning. Every submission runs against multiple test cases in the actual q interpreter. Working through a whole bank is the point, so every problem page now has a **Next** button that walks straight to the following problem in the list.



A HackerRank-style problem page — codepad on the right, Submit and Next below it.

Results: tracking progress by topic, not just overall

This is where it stopped being "a quiz" and started being a practice tool. Beyond an overall accuracy trend and a pass/fail breakdown by section, the dashboard now breaks the Multiple Choice bank down by *category* — attributes, joins, tables, ipc, and so on — so it's obvious which topic is actually dragging accuracy down, instead of one aggregate number.



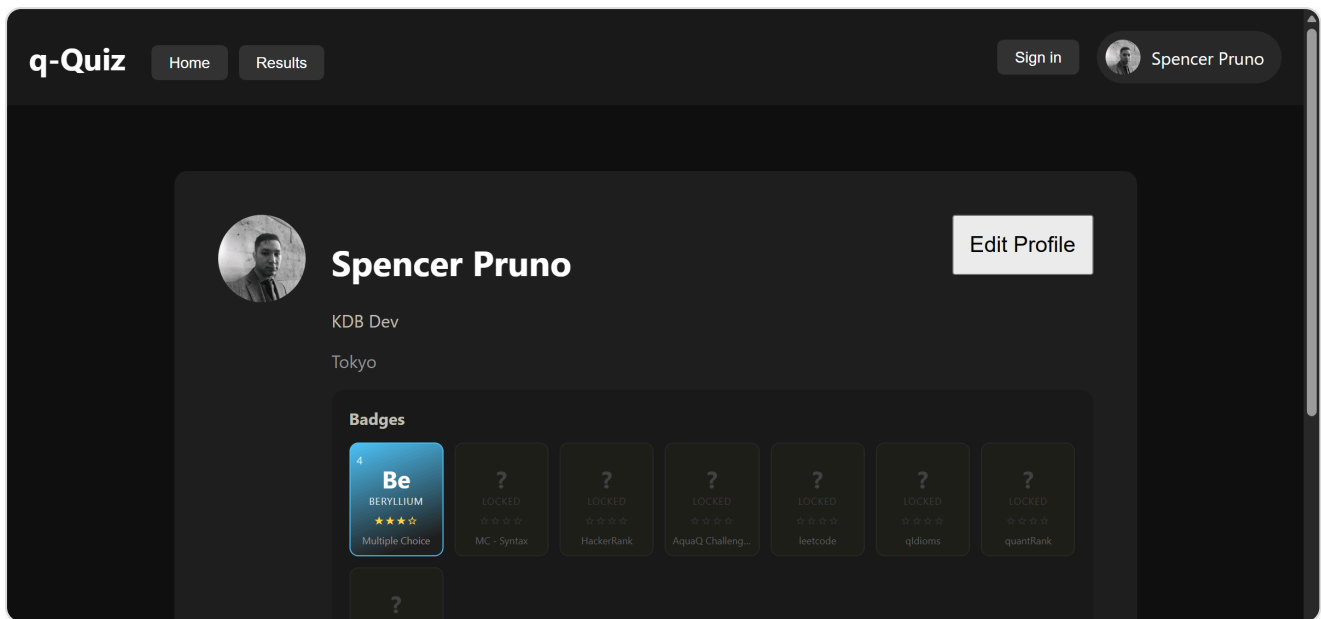
Attempt counts per section, plus accuracy over time.



Correct-vs-incorrect broken down by topic — split cleanly between the core and Syntax tracks.

Badges: gamification, periodic-table style

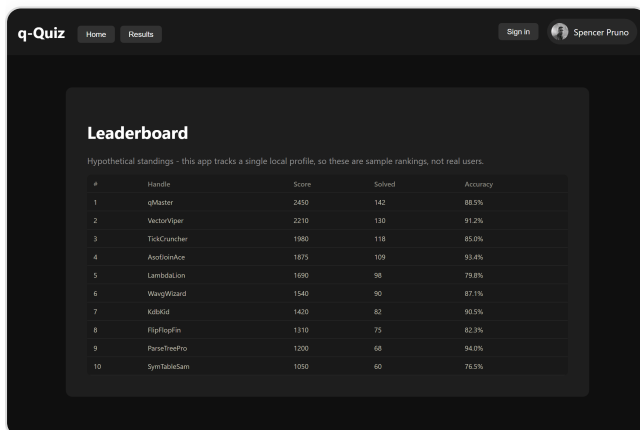
Progress tracking got a small game layer too — a badge per category, styled after the periodic table (atomic number, symbol, element name), that levels up every 50 correct answers (10 for the lower-volume challenge sections) and rolls onto the next element every four stars. Small feature, but it's a nice at-a-glance read on where the reps have actually gone in.



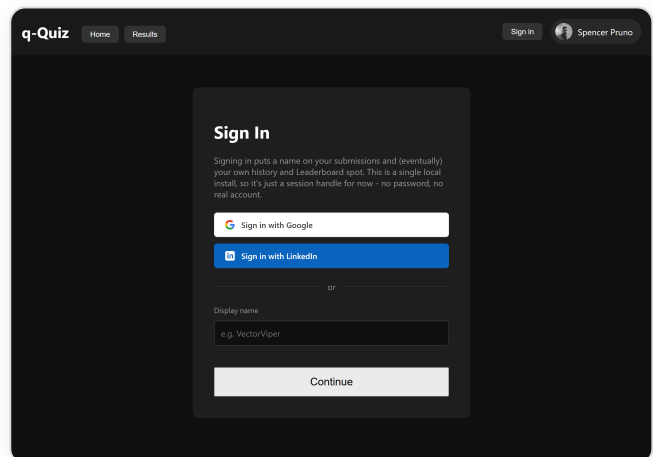
Badges unlock per category as correct answers accumulate — four stars, then the next element.

Community: Leaderboard, Jobs, and sign-in

Rounding it out: a Leaderboard, a Jobs board for kdb+/q roles, and a lightweight sign-in flow — Google and LinkedIn buttons are wired up and ready, with a display-name fallback live today — so progress and standings can eventually attach to a real identity instead of one shared local profile.



Leaderboard



Sign-in, with a Google/LinkedIn stub and a session fallback

Under the hood

q-Quiz is a Flask front end talking directly to a persistent kdb+/q process over IPC — every quiz answer and every codepad submission is graded by the real interpreter, not a lookup table. That single constraint shaped almost every design decision in the app, and it's exactly what makes the grading trustworthy.

It's still very much a work in progress — this write-up is a snapshot of something that's growing week to week. If you work with kdb+/q, or you're trying to learn it, I'd love to hear what you'd want to see in something like this.

[#kdb](#) [#kdbplus](#) [#qlang](#) [#fintech](#) [#softwareengineering](#) [#quant](#)